



The Great EIA Camel vs. Dwarf Racing System

Java REST API, Graphical User Interface, Security, Database and Docker

Backend Development Project

Universidad EIA

Instructor: Sebastián Zapata Ramírez



Academic warning: this story is fictional, intentionally absurd and designed exclusively for educational purposes. No camels, dwarfs or spreadsheets should be harmed during development.



1. The Completely Reliable Historical Background

Forty-seven years ago, the founders of Universidad EIA were working from the small university campus located in Zúñiga, Envigado. According to a recently discovered document of extremely questionable historical accuracy, the founders became bored with traditional academic activities such as teaching engineering, reviewing equations and asking students to correct indentation errors.

They therefore decided to create a slightly illegal and absolutely ridiculous business: Camel vs. Dwarf Racing.

The original competition was simple: one camel competed against a team of five dwarfs over a one-kilometer track located on the land where the Universidad EIA campus in Alto de Las Palmas stands today.

- The camel started whenever it felt emotionally prepared.
- The five dwarfs could use teamwork, strategy and dramatic motivational speeches.
- Nobody knew exactly who measured the kilometer.
- The winner received a mysterious golden trophy.
- The losers had to attend an eight-hour lecture about software documentation.

For many years, these races remained forgotten. However, Mr. Abandonado, a mysterious businessman, amateur sports commentator and self-proclaimed “Chief Executive Officer of Unnecessary Ideas,” has decided to bring the competition back.

- Dwarf vs. dwarf.
- Camel vs. dwarf.
- Dwarf vs. medium-sized competitor.
- Camel vs. camel.
- Individual competitors vs. teams.
- Any other configuration approved by the race administrators.

Mr. Abandonado attempted to manage the first race using an Excel file named `final_races_v2_FINAL_NOW_THIS_ONE.xlsx`. The file was accidentally deleted after someone sorted a column incorrectly. Your team has therefore been hired to build a secure, usable, database-backed and containerized system. The future of the competition, the reputation of Universidad EIA and several confused camels now depend on your software.

2. General Objective

Develop a complete information system for the fictional EIA Camel vs. Dwarf Racing League. The solution must include a Java backend exposed as a REST API and a graphical user interface that consumes that API. It must store information persistently, enforce authentication and authorization, validate business rules, handle errors correctly and run through Docker containers.

- Object-oriented programming.
- REST API design.
- Layered software architecture.
- Database modeling and persistence.



- Authentication and role-based authorization.
- Graphical user interface development.
- Client-server integration.
- Input validation and exception handling.
- Automated testing.
- Docker containerization.
- Version control using Git and GitHub.

3. Mandatory Technology Requirements

3.1 Backend

- Java 21 or later.
- Spring Boot is recommended.
- Spring Web, Spring Data JPA, Spring Validation and Spring Security are recommended.
- Another Java framework requires prior instructor approval.

3.2 Database

The team may use PostgreSQL, MySQL, MariaDB, MongoDB or another approved database. The selected database must run in a Docker container. H2 may be used only for automated tests or temporary local development; it is not acceptable as the final persistent database.

3.3 Graphical User Interface

The project must include a functional graphical user interface. It may be a web, desktop or mobile client, provided that it consumes the REST API and does not access the database directly.

- Recommended web options: React, Angular, Vue, Thymeleaf or another approved framework.
- Recommended desktop options: JavaFX or Swing.
- The interface must clearly show loading, success, empty and error states.
- The interface must adapt its visible actions according to the authenticated user role.
- A collection of Postman requests does not count as a graphical user interface.

3.4 Containerization

- A Dockerfile for the Java backend.
- A Dockerfile for the graphical client when applicable.
- A compose.yml file.
- A container for the backend.
- A container for the database.
- A container for the frontend when it is a separately deployed application.
- Any additional container required by the selected security solution.

The complete solution must start with one command:

```
docker compose up -d
```



4. Required Architecture

The backend must follow a clear layered architecture. At minimum, it must contain packages equivalent to:

controller		
service		
repository		
model	or	entity
dto		
security		
exception		
config		

4.1 Backend responsibilities

- Controller: receives HTTP requests, validates input format, invokes services and returns HTTP responses. Controllers must not contain business logic.
- Service: implements business rules, race logic, registrations, statistics and coordination between repositories.
- Repository: performs database access and persistence.
- DTO: separates API contracts from persistence entities. Entities must not be exposed directly.
- Exception: centralizes error handling and produces consistent error responses.

4.2 Frontend responsibilities

- Views or pages: present information and user actions.
- Components: reusable visual elements such as forms, tables, cards, dialogs and notifications.
- API client or service layer: communicates with the backend.
- State management: stores session, user role and relevant screen data.
- Route protection: prevents unauthorized users from opening restricted pages.
- Frontend validation: improves user experience but does not replace backend validation.

5. Mandatory Functional Modules

The system must contain at least seven clearly identifiable modules. The following modules are mandatory.



Module 1 — Authentication and Security

Security may be implemented with Auth0, Keycloak, Amazon Cognito, JWT with Spring Security, or another approved identity provider.

Required roles

Role	Minimum permissions
Administrator	Manage users, competitors, teams, races, registrations, results and audit records.
Race Organizer	Manage races, registrations and results; view competitors and teams.
Viewer	Read public information, schedules, results and standings only.

Security requirements

- User login and logout.
- Secure password storage when authentication is implemented manually.
- BCrypt or another secure password-hashing algorithm.
- Token-based authentication and token expiration.
- Role-based authorization in the backend.
- Route and action protection in the graphical interface.
- 401 Unauthorized for missing or invalid authentication.
- 403 Forbidden for authenticated users without permission.
- Passwords, tokens and secrets must never be returned by the API or committed to GitHub.
- Sensitive values must use environment variables, Docker secrets or a local .env file excluded by .gitignore.

Suggested endpoints

POST	/api/auth/register
POST	/api/auth/login
POST	/api/auth/refresh
GET	/api/auth/profile

Module 2 — Competitor Management

A competitor may be a dwarf, camel, medium-sized competitor or another fictional category approved by an administrator.

Required data

- Identifier, name and unique nickname.
- Competitor type.
- Date of birth or approximate age.
- Weight and height.
- Country or place of origin.
- Current status and registration date.
- Optional team.



- Victories, defeats and completed races.

Minimum enumerations

CompetitorType:	DWARF,	CAMEL,	MEDIUM,	OTHER
CompetitorStatus:	ACTIVE, INJURED, SUSPENDED, RETIRED			

Business rules

- Name cannot be empty.
- Weight and height must be positive.
- Nickname must be unique.
- Only ACTIVE competitors may be registered in new races.
- A competitor with official results cannot be physically deleted; it must be retired or deactivated.
- A camel must not be registered as a dwarf, even if it strongly believes in itself.

Suggested endpoints

POST	/api/competitors
GET	/api/competitors
GET	/api/competitors/{id}
PUT	/api/competitors/{id}
PATCH	/api/competitors/{id}/status
DELETE	/api/competitors/{id}

Filtering, pagination and sorting are mandatory.

Module 3 — Team Management

Required data

- Identifier, unique team name and description.
- Creation date and status.
- Coach or responsible person.
- List of competitors.
- Victories and defeats.

Business rules

- A team must contain at least one competitor before entering a race.
- A competitor cannot belong to more than one active team at the same time.
- A suspended team cannot enter a race.
- A competitor cannot be added twice to the same team.
- A team with official race history cannot be deleted; it must be deactivated.
- The maximum number of members must be configurable.

Suggested endpoints

POST	/api/teams
GET	/api/teams
GET	/api/teams/{id}
PUT	/api/teams/{id}



DELETE	/api/teams/{id}
POST	/api/teams/{teamId}/members/{competitorId}
DELETE	/api/teams/{teamId}/members/{competitorId}

Module 4 — Race Management

Required data

- Identifier, race name and description.
- Scheduled date and time.
- Starting and finishing locations.
- Distance in meters.
- Maximum number of participants.
- Race type and race status.
- Organizer and registration deadline.
- Creation and last-modification timestamps.

RaceType:	INDIVIDUAL,	TEAM,	MIXED
RaceStatus:	DRAFT, OPEN_FOR_REGISTRATION,	CLOSED_FOR_REGISTRATION,	IN_PROGRESS, COMPLETED, CANCELLED

Business rules

- Distance must be greater than zero.
- A race cannot be created in the past.
- The registration deadline must be earlier than the race start time.
- A completed race cannot be edited.
- A cancelled race cannot receive registrations.
- At least two valid participants are required to start.
- Capacity cannot be exceeded.
- A race cannot be completed without official results.
- A completed race cannot magically return to DRAFT, even if the camel requests a rematch.

Suggested endpoints

POST	/api/races
GET	/api/races
GET	/api/races/{id}
PUT	/api/races/{id}
PATCH	/api/races/{id}/status
DELETE	/api/races/{id}

Module 5 — Race Registration

Required data

- Identifier and race.
- Competitor or team.
- Registration date and status.
- Assigned lane or starting position.



- Validation notes.
- User who performed the registration.

RegistrationStatus: PENDING, APPROVED, REJECTED, CANCELLED

Business rules

- Registration is allowed only while the race is open and before the deadline.
- A competitor or team cannot be registered twice in the same race.
- All individual competitors and team members must be eligible.
- A participant cannot compete simultaneously as an individual and as a team member in the same race.
- Registration type must match race type.
- Starting positions cannot be duplicated.
- Rejected registrations must include a clear reason.

Suggested endpoints

POST	/api/races/{raceId}/registrations
GET	/api/races/{raceId}/registrations
GET	/api/registrations/{id}
PATCH	/api/registrations/{id}/approve
PATCH	/api/registrations/{id}/reject
DELETE	/api/registrations/{id}

Module 6 — Results and Standings

Required data

- Race and participant.
- Starting and final position.
- Completion time and penalty time.
- Result status and notes.
- User who recorded the result and recording timestamp.

ResultStatus: FINISHED, DISQUALIFIED, DID_NOT_FINISH, DID_NOT_START

Business rules

- Results may only be entered for an IN_PROGRESS race.
- Only approved participants may receive results.
- Final positions cannot be duplicated among normal finishers.
- Completion time must be positive.
- A disqualified participant cannot win.
- Only one official winner is allowed.
- Updating a result must update statistics consistently.
- The system must prevent two winners, three second places and a camel finishing before the race began.

Suggested endpoints

POST	/api/races/{raceId}/results
GET	/api/races/{raceId}/results



PUT	/api/results/{id}
GET	/api/results/{id}
GET	/api/standings
GET	/api/standings/competitors
GET	/api/standings/teams

Position	Points
First	10
Second	7
Third	5
Fourth	3
Fifth	1
Did not finish	0
Disqualified	0

Module 7 — Graphical User Interface

The graphical interface is a mandatory functional module, not a decorative extra. It must allow users to perform the actions permitted by their roles through clear screens and forms.

Minimum required screens

- Login screen.
- Main dashboard with a summary of upcoming races, active competitors and recent results.
- Competitor list with search, filters, pagination and detail view.
- Competitor creation and editing form for authorized roles.
- Team list, team detail and member-management screen.
- Race list and race detail screen.
- Race creation and editing form.
- Registration-management screen.
- Result-entry screen.
- Standings or leaderboard screen.
- User profile and logout option.
- Access-denied and not-found screens.

Interface behavior

- Buttons and menu options must be hidden or disabled when the role lacks permission.
- The client must store and send the authentication token securely.
- Forms must show field-level validation messages.
- API errors must be translated into understandable messages for users.
- A confirmation dialog is required for destructive actions.
- Lists must show useful empty states instead of blank screens.
- Long operations must show a loading indicator.
- Success and failure notifications must be visible.
- The interface must not expose stack traces, passwords or internal database identifiers unnecessarily.
- The user must be able to complete the main demonstration flow without using Postman.



Usability expectations

- Consistent navigation and visual hierarchy.
- Readable typography and sufficient contrast.
- Labels for all form fields.
- Keyboard-accessible forms when using a web interface.
- Responsive behavior is recommended for web applications.
- The interface should look like a coherent product, not seven unrelated HTML pages assembled five minutes before the deadline.

Module 8 — Audit Log

The system must record important actions such as login, user creation, competitor changes, race cancellation, registration decisions and result modifications.

- Identifier.
- User.
- Action.
- Entity type and entity identifier.
- Date and time.
- Optional description.
- Optional previous and new values.

Only administrators may view the complete audit log.

6. REST API Design Requirements

- Use GET to retrieve, POST to create, PUT to fully update, PATCH for partial updates or state transitions, and DELETE to remove or deactivate.
- Return 201 Created when a resource is created.
- Return 204 No Content when a delete operation succeeds without a response body.
- Use 400 for invalid requests, 401 for missing authentication, 403 for insufficient permissions, 404 for missing resources and 409 for business conflicts.
- Use a consistent JSON naming convention.
- All endpoints must be testable independently of the graphical interface.

Structured error response

```
{
  "timestamp": "2026-08-15T14:30:00",
  "status": 404,
  "error": "Not Found",
  "message": "Competitor with ID 45 was not found",
  "path": "/api/competitors/45"
}
```



7. Validation Requirements

Backend validation is mandatory. Frontend validation is also mandatory for usability, but it never replaces backend checks.

- Use appropriate validation annotations such as @NotNull, @NotBlank, @Size, @Positive, @Email, @Past and @Future.
- Return understandable field-level validation messages.
- Never return raw stack traces to the user.
- Validate identifiers, state transitions, permissions and cross-entity business rules.

8. Database Requirements

The data model should include entities equivalent to User, Role, Competitor, Team, TeamMember, Race, RaceRegistration, RaceResult and AuditLog.

- Primary keys and foreign keys or equivalent references.
- Unique constraints.
- Required fields.
- Appropriate indexes.
- Referential integrity.
- Persistent named Docker volume.
- Entity-relationship diagram.

Minimum initial data

- One administrator, one organizer and one viewer.
- Five dwarfs, two camels and two medium-sized competitors.
- Two teams.
- Three races in different statuses.
- At least one completed race with results.

9. Testing Requirements

The project must include at least 15 meaningful automated tests. Tests that only verify getters and setters will not count.

- Create a valid competitor.
- Reject a competitor with invalid weight.
- Reject a duplicated nickname.
- Create a valid race.
- Reject a race scheduled in the past.
- Register an active competitor successfully.
- Reject a suspended competitor.
- Reject a duplicated registration.



- Reject registration after the deadline.
- Record a valid result.
- Reject two winners in one race.
- Prevent a viewer from creating a race.
- Allow an administrator to create a race.
- Return 401 without a valid token.
- Return 404 for a missing resource.

Frontend tests are optional but recommended. At minimum, the team must demonstrate that the interface correctly handles successful responses, validation errors and authorization failures.

10. Docker Requirements

- Application container.
- Database container.
- Frontend container when applicable.
- Named database volume.
- Isolated Docker network.
- Environment variables.
- Port configuration.
- Automatic startup.
- Health checks are recommended.

```
DB_HOST
DB_PORT
DB_NAME
DB_USERNAME
DB_PASSWORD
JWT_SECRET
JWT_EXPIRATION
API_BASE_URL
```

The repository may include a `.env.example` file, but it must never include real credentials.

11. Git and GitHub Requirements

- Complete project in a GitHub repository.
- Meaningful commits from every team member.
- Branch usage and pull requests.
- `.gitignore`.
- `README.md`.
- Database diagram.
- Docker instructions.
- Test instructions.
- Clear contribution history.



```
main
develop
feature/security
feature/competitors
feature/races
feature/frontend
feature/results
```

A repository containing only one final commit named final will be treated with the same enthusiasm as a camel attending a JavaScript convention.

12. README Requirements

- Project name and description.
- Team members.
- Architecture explanation.
- Technologies used.
- Database model.
- Security strategy.
- User roles and permissions.
- Backend and frontend installation instructions.
- Docker execution instructions.
- Environment variables.
- URLs and ports for each component.
- Testing instructions.
- Sample users.
- Sample API requests.
- Known limitations.
- Future improvements.

13. Optional Bonus Features

- GitHub Actions pipeline.
- Automatic test execution and Docker image build.
- Cloud deployment.
- Refresh tokens.
- Email notifications.
- Live race updates with WebSockets.
- Redis caching.
- Rate limiting.
- Flyway or Liquibase migrations.
- Testcontainers.
- CSV or PDF result export.



- Competitor profile images.
- Metrics and observability.
- Soft delete.
- Optimistic locking.
- Idempotency keys.

Bonus features do not replace missing mandatory requirements. A project with Kubernetes, artificial intelligence and satellite telemetry but without authentication or a usable interface is still incomplete.

14. Deliverables

- GitHub repository with complete source code.
- Dockerized backend, database and graphical interface.
- Entity-relationship diagram.
- Postman or Insomnia collection for backend verification.
- Technical report describing architecture, data model, security, interface, business rules, testing and Docker.
- Demonstration video lasting between 8 and 12 minutes.

Video demonstration requirements

- All team members must participate.
- Show login and role restrictions.
- Create a competitor and a team.
- Create a race and register participants.
- Record results and show standings.
- Demonstrate at least two handled errors.
- Show the graphical interface performing the main workflow.
- Show the containers running.
- Show automated tests executing.
- The video must not consist exclusively of reading slides.

15. Suggested Demonstration Scenario

- An administrator logs in through the graphical interface.
- The administrator creates a camel named Byte.
- Five dwarfs are created: Null Pointer, Stack Overflow, Little Lambda, Captain Cache and Tiny Docker.
- The dwarfs are added to The Five Exceptions team.
- An organizer creates a one-kilometer mixed race.
- Byte and The Five Exceptions register.
- A viewer attempts to modify the race and receives an access-denied message.
- The organizer starts the race.
- Results are recorded through the interface.
- The standings are updated.



Universidad EIA | Backend Development Project

- The audit log records the important actions.

Historical sources indicate that Tiny Docker stopped halfway through the race because “it worked on his machine.”



16. Evaluation Criteria

Criterion	Percentage
Functional requirements and business rules	22%
Graphical user interface and user experience	15%
API design and code organization	13%
Authentication and authorization	15%
Database design and persistence	10%
Docker and execution environment	10%
Automated testing	8%
Documentation, GitHub workflow and team participation	4%
Final demonstration and explanation	3%
Total	100%

Important grading conditions

- The project does not run.
- The graphical interface is missing or cannot complete the main workflow.
- The interface accesses the database directly instead of using the API.
- The database is not persistent.
- The application is not dockerized.
- Authentication or role enforcement is missing.
- Business rules exist only in slides but not in code.
- Errors return stack traces.
- The repository contains passwords or secrets.
- There are no meaningful automated tests.
- Only one team member understands the application.
- The application works exclusively on the computer of “the teammate who is not here today.”

17. Final Warning from Mr. Abandonado

Mr. Abandonado expects a system that is secure, organized, usable and reliable. He does not accept explanations such as “the database was working yesterday,” “Docker deleted everything,” “the token expired during the presentation,” “the camel changed the environment variables,” or “the interface works, but only if you click the buttons in the correct astrological order.”

A successful system must do more than return 200 OK. It must protect information, enforce business rules, provide a usable interface, handle errors correctly and survive contact with users, databases, containers and unpredictable racing camels.

Good luck. May your containers remain healthy, your tokens remain valid and your camels respect the API contract.